WEBRUM-99: Best Practice Summary

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 10 of 10 | **Created:** March 2026 | **Last Updated:** 10/19/2026

Overview

This notebook consolidates every actionable best practice from the WEBRUM series (notebooks 01-08) into a single definitive reference. Each practice specifies the exact setting, value, or action to take — no hedging, no "it depends." Practices are organized by category and prioritized as Critical, Recommended, or Optional.

Table of Contents

1. RUM Agent Configuration
2. SPA Instrumentation
3. Core Web Vitals Thresholds
4. Performance Optimization
5. Error Monitoring
6. Session Replay and Privacy
7. Session Analysis
8. Dashboards and Alerting
9. DQL Query Patterns

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	At least one web application with RUM injection active
Permissions	`storage:events:read`, `storage:metrics:read`, `storage:entities:read`
Previous Notebooks	WEBRUM-01 through WEBRUM-08 (for full context)

1. RUM Agent Configuration

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use automatic injection	Injection method: **Automatic** (OneAgent modifies HTML responses at the web server)	Critical	WEBRUM-01
2	Filter out bots from all RUM queries	Always apply `filter dt.rum.user_type == "real_user"` in every DQL query against `user.sessions`	Critical	WEBRUM-01

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Use manual injection for CDN/static-hosted apps	Add RUM `<script>` tag to `<head>` before all other scripts when no server-side OneAgent exists	Recommended	WEBRUM-02
4	Use agentless RUM only as last resort	Select agentless monitoring only when OneAgent cannot be deployed at all	Optional	WEBRUM-02
5	Set session timeout to 30 minutes	Keep the default 30-minute inactivity timeout for session boundaries	Recommended	WEBRUM-01
6	Enable Fetch API monitoring	**Settings > Web and mobile monitoring > Async web requests and SPAs > Fetch requests**: **Enabled**	Critical	WEBRUM-02
7	Deploy RUM + Synthetic together	Use synthetic for availability SLAs and baselines; use RUM for real user experience	Critical	WEBRUM-01, WEBRUM-08
7a	Tag RUM data with OpenPipeline DQL processors	Processor on `frontend.name` with `isNull(primary_tags.<key>)` in all six RUM scopes; `arrayContains(frontend.name, …)` in User sessions	Recommended	WEBRUM-01

2. SPA Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Enable SPA route change capture	**Settings > Web and mobile monitoring > Async web requests and SPAs > SPA route changes**: **Enabled**	Critical	WEBRUM-02
9	Enable Async Web Requests capture	**Settings > Web and mobile monitoring > Async web requests and SPAs > Async web requests**: **Enabled**	Critical	WEBRUM-02
10	Configure user action naming rules for dynamic URLs	Replace dynamic segments: `/users/{id}` becomes `/users/{*}` via **Settings > Web and mobile monitoring > User action naming**	Critical	WEBRUM-02
11	Use `data-dtname` attribute on interactive elements	Add `data-dtname="Checkout"` to buttons/links for meaningful action names	Recommended	WEBRUM-02
12	Load RUM agent before the SPA router initializes	Place the RUM `<script>` tag in `<head>` before framework bootstrap scripts	Critical	WEBRUM-02
13	Use `dtrum.enterAction()` / `dtrum.leaveAction()` for custom actions	Wrap business-critical interactions in manual RUM actions when auto-detection is insufficient	Recommended	WEBRUM-02
14	Validate SPA instrumentation after setup	Query for `user_action.type == "soft_navigation"` (route changes) and `"same_view"` (XHR) actions — if an SPA shows only `hard_navigation`, SPA monitoring is misconfigured	Critical	WEBRUM-02
15	Separate mobile WebView and desktop into distinct RUM apps	Create two Dynatrace applications; use application detection rules based on User-Agent string	Recommended	WEBRUM-02
16	Add custom User-Agent suffix for WebView detection	Android: `setUserAgentString()` / iOS: `applicationNameForUserAgent` with app name + version	Recommended	WEBRUM-02
17	Enable hybrid WebView monitoring in mobile SDK	Android: `hybridWebView { enabled true; domains "your-domain.com" }` / iOS: `DTXHybridApplication = true`	Recommended	WEBRUM-02

#	Best Practice	Recommended Setting/Value	Priority	Source
18	Never instrument third-party WebViews	Only add your own domains to `domains` / `DTXMonitoredDomains` — never OAuth, payment, or external pages	Critical	WEBRUM-02

3. Core Web Vitals Thresholds

#	Best Practice	Recommended Setting/Value	Priority	Source
19	Target LCP at or below 2.5 seconds	**Largest Contentful Paint p75 <= 2500ms**. Poor threshold: > 4000ms	Critical	WEBRUM-03
20	Target INP at or below 200 milliseconds	**Interaction to Next Paint p75 <= 200ms** on `web_vitals.interaction_to_next_paint`. Poor threshold: > 500ms. `inp.status == "reported"` does not mean slow	Critical	WEBRUM-03
21	Target CLS at or below 0.1	**Cumulative Layout Shift p75 <= 0.1**. Poor threshold: > 0.25	Critical	WEBRUM-03
22	Use p75 (not average) for CWV evaluation	Google uses 75th percentile as the pass/fail threshold for CWV	Critical	WEBRUM-03
23	Track CWV per page, not just per app	Aggregate CWV by `page.detected_name` to identify which specific pages fail thresholds	Recommended	WEBRUM-03
24	Set explicit `width` and `height` on all images	Prevents CLS from images loading without reserved space	Critical	WEBRUM-03
25	Use `font-display: swap` with fallback font metrics	Prevents CLS from FOUT (Flash of Unstyled Text) during web font loading	Recommended	WEBRUM-03
26	Use fixed dimensions on ads, embeds, and iframes	Set explicit `width` / `height` on all third-party content containers to prevent layout shifts	Recommended	WEBRUM-03
27	Break long JavaScript tasks to improve INP	Split tasks > 50ms using `requestIdleCallback`, `setTimeout(0)`, or web workers	Critical	WEBRUM-03
28	Batch DOM reads and writes to reduce INP	Avoid forced layout/reflow by separating DOM read and write operations	Recommended	WEBRUM-03
29	Build a CWV scorecard query	Use the single-query pattern from WEBRUM-03 to show % Good/NI/Poor for all three metrics	Recommended	WEBRUM-03
30	Track CWV trends at hourly (LCP/INP) and daily (CLS) granularity	Use `makeTimeseries` with `interval:1h` for LCP/INP, `interval:24h` for CLS	Recommended	WEBRUM-03

4. Performance Optimization

#	Best Practice	Recommended Setting/Value	Priority	Source
31	Target TTFB at or below 800 milliseconds	**Time to First Byte p75 <= 800ms** (Google recommendation), on `coalesce(web_vitals.time_to_first_byte / 1ms, ttfb.value)` — not `ttfb.waiting_duration`	Critical	WEBRUM-06

#	Best Practice	Recommended Setting/Value	Priority	Source
32	Use DNS prefetching for third-party domains	Add `<link rel="dns-prefetch" href="//cdn.example.com">` for external resources	Recommended	WEBRUM-06
33	Enable HTTP/2 on all web servers	Reduces TCP connection overhead; multiplexes requests on a single connection	Critical	WEBRUM-06
34	Enable TLS 1.3 with OCSP stapling	Reduces SSL handshake time by one round trip	Recommended	WEBRUM-06
35	Defer non-critical JavaScript	Use `defer` or `async` attributes on `<script>` tags that are not needed for initial render	Critical	WEBRUM-06
36	Lazy-load below-the-fold images	Use `loading="lazy"` on `<img>` tags not visible in the initial viewport	Recommended	WEBRUM-06
37	Monitor the DOM Interactive to Load Event gap	A large gap indicates excessive sub-resource loading; target < 2 seconds	Recommended	WEBRUM-06
38	Deploy CDN edge servers for high-TTFB regions	If TTFB is high for specific geographies, add CDN nodes closer to those users	Critical	WEBRUM-06
39	Prioritize slow pages by impact score	Calculate `impact_score = page_views * avg_load_ms` (from `performance.load_event_end` on navigation events) and fix highest-impact pages first	Recommended	WEBRUM-06
40	Set page load performance SLA at 3 seconds	Track `% of page loads with performance.load_event_end <= 3s` on navigation events as primary performance SLA — never page-summary `duration`, which is how long the page was open	Critical	WEBRUM-08

5. Error Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Source
41	Target error session rate below 2%	**% of sessions with error.count > 0 < 2%**	Critical	WEBRUM-05, WEBRUM-08
42	Rank errors by affected session count, not occurrence count	A retry loop in one session inflates occurrence count; `countDistinct(dt.rum.session.id)` is the true impact metric	Critical	WEBRUM-05
43	Monitor XHR/fetch errors separately from JS errors	Filter `characteristics.has_failed_request == true` (or `error.type == "request"`) to isolate backend API failures	Recommended	WEBRUM-05
44	Find frustrated users through replay, not a rage-click field	Rage clicks are a RUM Classic signal with no documented New RUM field; review error-impacted sessions in Session Replay	Recommended	WEBRUM-05, WEBRUM-07
45	Correlate errors with session outcomes	Compare bounce rate and avg actions for "With Errors" vs "No Errors" sessions to quantify error impact	Recommended	WEBRUM-05
46	Segment error rates by browser family	Query `error.count > 0` grouped by `browser.name` to identify browser-specific issues	Recommended	WEBRUM-05

#	Best Practice	Recommended Setting/Value	Priority	Source
47	Alert on new error messages	Query `min(start_time)` per error group over 7 days and keep groups first seen in the last hour — `user.events` has no populated `timestamp`	Critical	WEBRUM-08
48	Use `dtrum.reportError()` for custom business errors	Report application-specific errors (e.g., payment validation failures) via the RUM API	Recommended	WEBRUM-05

6. Session Replay and Privacy

#	Best Practice	Recommended Setting/Value	Priority	Source
49	Set Session Replay default masking to "Mask user input"	**Settings > Session Replay > Data privacy > Default masking level**: **Mask user input**	Critical	WEBRUM-07
50	Block entire PII containers with `data-dtrum-block="true"`	Add `data-dtrum-block="true"` to any `<div>` containing sensitive data (SSN, CC, health info)	Critical	WEBRUM-07
51	Add CSS masking rules for all form inputs with sensitive data	**Settings > Session Replay > Data privacy**: add selectors like `.credit-card-form input`, `[data-sensitive]`	Critical	WEBRUM-07
52	Set replay sampling rate based on traffic volume	High-traffic production: **1-5%**; internal apps: **10-25%**; pre-prod: **50-100%**	Recommended	WEBRUM-07
53	Enable error-triggered replay capture	Configure conditional capture: automatically record sessions when JavaScript errors occur	Recommended	WEBRUM-07
54	Enable frustration-triggered replay capture	Configure conditional capture for sessions with rage clicks or exit intent	Recommended	WEBRUM-07
55	Err on over-masking, not under-masking	It is better to mask non-sensitive content than to accidentally capture PII	Critical	WEBRUM-07
56	Conduct a privacy review before enabling Session Replay in production	Review all captured content against GDPR, CCPA, and HIPAA requirements before go-live	Critical	WEBRUM-07
57	Use Dynatrace native replay over third-party tools	Native replay integrates with performance waterfall, backend traces, and Dynatrace Intelligence; third-party tools lack this correlation	Recommended	WEBRUM-07
58	Follow the replay investigation workflow	1) Identify via DQL, 2) Watch replay, 3) Correlate with waterfall, 4) Note pattern, 5) Quantify with DQL	Recommended	WEBRUM-07

7. Session Analysis

#	Best Practice	Recommended Setting/Value	Priority	Source
59	Define custom session properties for business context	Configure **Settings > Web and mobile monitoring > Session and user action properties** with: `session.user_id`, `session.plan_type`, `session.cart_value`, `session.ab_variant`	Critical	WEBRUM-04
60	Target bounce rate below 40%	**% of sessions with user_action_count == 1 < 40%**	Recommended	WEBRUM-04, WEBRUM-08
61	Segment sessions by engagement level	Bucket sessions: No actions (0), Bounce (1 action), Low (2-3), Medium (4-10), High (10+)	Recommended	WEBRUM-04

#	Best Practice	Recommended Setting/Value	Priority	Source
62	Track entry and exit pages	Use `takeFirst(page.detected_name)` and `takeLast(page.detected_name)` on page summaries grouped by `dt.rum.session.id` to map user journeys	Recommended	WEBRUM-04
63	Use session properties for conversion tracking, not URL pattern matching	Mark conversions via session properties instead of fragile `contains(page.detected_name, "confirmation")` patterns	Critical	WEBRUM-04
64	Analyze sessions by hour of day for capacity planning	Group sessions by `getHour(start_time)` over 7 days to identify peak traffic windows	Optional	WEBRUM-04
65	Segment by geography and device	Break down sessions by `geo.country.iso_code`, `browser.name` and `os.name` to find regional/device issues (web RUM has no connection-type field)	Recommended	WEBRUM-04, WEBRUM-06

8. Dashboards and Alerting

#	Best Practice	Recommended Setting/Value	Priority	Source
66	Set Apdex threshold T = 3 seconds for page loads	Satisfied: <= 3s, Tolerating: 3-12s, Frustrated: > 12s — on `performance.load_event_end` of navigation events	Critical	WEBRUM-08
67	Set Apdex threshold T = 2.5 seconds for XHR and route changes	Satisfied: <= 2.5s, Tolerating: 2.5-10s, Frustrated: > 10s	Critical	WEBRUM-08
68	Target Apdex score above 0.85	Apdex 0.85-0.93 = Good; 0.94-1.00 = Excellent; below 0.70 = investigate immediately	Critical	WEBRUM-08
69	Build an executive dashboard with 6 KPIs	Tiles: Apdex, session count, error rate %, bounce rate %, CWV pass rate %, avg session duration	Recommended	WEBRUM-08
70	Build an operational dashboard with real-time error view	Tiles: error count per 15m by type, active error table (last 1h), performance SLA % under 3s	Recommended	WEBRUM-08
71	Alert on error rate spike	Threshold: **> 5% of sessions with errors** sustained for **15 minutes**	Critical	WEBRUM-08
72	Alert on Apdex drop	Threshold: **Apdex < 0.7** sustained for **15 minutes**	Critical	WEBRUM-08
73	Alert on performance degradation	Threshold: **p75 page load (`performance.load_event_end`) > 5 seconds** sustained for **15 minutes**	Critical	WEBRUM-08
74	Alert on traffic anomaly	Threshold: **session count < 50% of previous day's hourly average**	Recommended	WEBRUM-08
75	Alert on CWV regression	Threshold: **LCP p75 > 4 seconds** sustained for **30 minutes**	Recommended	WEBRUM-08
76	Use metric event evaluation window of 15 minutes	**Settings > Anomaly detection > Metric events**: evaluation window **15m**, sliding window **5m**	Recommended	WEBRUM-08
77	Prefer Dynatrace Intelligence anomaly detection over static thresholds in production	Dynatrace Intelligence automatically baselines normal behavior, reducing false positives from seasonal patterns	Recommended	WEBRUM-08

#	Best Practice	Recommended Setting/Value	Priority	Source
78	Combine RUM and synthetic in a single dashboard	Use `append` to show side-by-side comparison: RUM real-user metrics vs synthetic clean-room metrics	Recommended	WEBRUM-08
79	Track CWV pass rate as executive KPI	Target: **> 75% of page loads meeting all three CWV thresholds**	Recommended	WEBRUM-08

> **Where new alerts are created (SaaS 1.344).** Row 76 configures the legacy metric-events surface; WEBRUM-08 recommends Davis anomaly detectors (custom alerts) for new RUM alerting. *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and WEBRUM-08 builds the modern path there.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

9. DQL Query Patterns

#	Best Practice	Recommended Setting/Value	Priority	Source
80	Always include `from:` time range on RUM queries	Use `from:-1h` for real-time, `from:-24h` for daily, `from:-7d` for trends	Critical	WEBRUM-01
81	Always filter `dt.rum.user_type == "real_user"` on session queries	Excludes bots (`robot`) and synthetic monitors (`synthetic`) from real-user analysis	Critical	WEBRUM-01
82	Use `isNotNull()` before aggregating optional fields	Apply `filter isNotNull(lcp_ms)` on `lcp_ms = coalesce(web_vitals.largest_contentful_paint / 1ms, lcp.start_time)` (or the metric you aggregate) before CWV aggregations	Critical	WEBRUM-03
83	Use `countDistinct(dt.rum.session.id)` for error impact measurement	Never use `count()` alone — it inflates impact from retry loops in single sessions	Critical	WEBRUM-05
84	Require minimum sample size before drawing conclusions	Use `filter page_views > 20` (or similar) to avoid misleading statistics from low-volume pages	Recommended	WEBRUM-03, WEBRUM-06
85	Use nanosecond-to-millisecond conversion: `/ 1ms`	`duration` -typed fields are nanosecond durations — divide by `1ms` / `1s` ; `lcp.start_time` , `ttfb.value` and `cls.value` are plain numbers already in ms (CLS unitless) — use them directly	Critical	WEBRUM-03
86	Alias all aggregations before using in `sort`	Always write `summarize c = count()` then `sort c desc` , never `sort count() desc`	Critical	All

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace.

Always verify information against official Dynatrace documentation.